Chapter 1

Deep Learning topics

1.1. Deep Q Networks (Reinforcement Learning)

1.1.1 Introduction:

Reinforcement Learning (RL) is a way of programming agents by reward and punishment without specifying how the task can be achieved. An *agent* is a learning algorithm that learns to make intelligent decisions in the environment(world of the agent). The agent stays within the *environment*. A *state* is a position or a moment in the environment that the agent represents. An *action* is a move made by the agent, which causes a status change in the environment. The agent and environment continuously interact with each other. At each timestep, the agent takes action on the environment based on its policy $\pi(a_t|s_t)$, where s_t is the current observation from the environment and receives a reward r_{t+1} and the subsequent observation s_{t+1} from the environment. The *reward* can be a positive or negative numerical value, depending on the action. The goal is to improve the policy to maximise the *cumulative sum* of rewards (return).

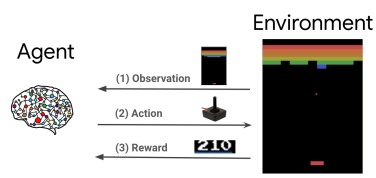

Figure 1.1: General Overview of the RL Problem[1]

Thus, RL follows a learning from experience kind of strategy. It presents several challenges from a deep learning point of view. Firstly, most deep learning applications require large amounts of hand labelled training data. On the other hand, RL algorithms must learn from a scalar reward signal. Also, most deep learning algorithms assume the data to be independent. In RL, states are highly correlated as they occur in sequence one after the other depending on the observation and reward. Furthermore, the data distribution changes as the algorithm learns new behaviours. Deep learning methods generally assume a fixed underlying distribution. The techniques employed in order to overcome these challenges are discussed in the upcoming sections.

1.1.2 Preliminary concepts:

Markov Decision Process (MDP):

A Markov Decision Process (MDP) is a 5-element tuple (S, A, P, R, γ), where:

- S is a set of states.
- A is a set of actions.
- $P(s'|s, a)$ is the probability that action a in state s at time t will lead to state s' at time $t + 1$. Each state in the environment is a consequence of its previous state, resulting from its previous state. Storing all this data

[1] Introduction to RL and Deep Q Networks- TensorFlow

is very difficult due to a large number of iterations. To resolve this in-feasibility, each state is assumed to follow the Markov property. It is a property of memorylessness, stating that each state depends only on the previous state and the transition to the current state.

- $R(s, a, s')$ is the immediate reward received after a transition from state s to s', due to action a.
- γ is the discounted factor which is used to generate a discounted reward, $\gamma \in [0, 1]$. Rewards are discounted as future rewards can be valued differently depending on the scenario. γ helps find the balance between the importance given to the immediate reward and the future rewards.

Bellman Equation:

The Bellman equation is used to solve the MDP, i.e. it helps find the optimal policy. The Bellman equation says that the value of a state can be found by taking the sum of the immediate reward and the discounted value of the next state. For a transition from state s to s' with reward r and due to action, the Bellman equation for the value function $V(s)$ can be expressed as:

$$V(s) = R(s, a, s') + \gamma V(s') \tag{1.1}$$

$V(s)$ is the value of the state s. The Bellman equation of the Q function states that the Q value of (s, a) can be obtained as a sum of the immediate reward r and the discounted Q value of the next pair (s', a'):

$$Q(s, a) = R(s, a, s') + \gamma Q(s', a') \tag{1.2}$$

Here, $Q(s', a')$ is the Q value of the next state-action pair.

ϵ-greedy Exploration:

Exploring the environment is a crucial factor in the convergence of the optimal policy. It helps in learning more about the environment and finding more strategies to achieve the goal. The opposite of exploration is exploitation, where the agent uses the existing strategies to reach the goal repeatedly. Here, the agent has incomplete information about the environment, as it only uses the already known strategies. The best strategy is to use a hybrid model of exploration as well as exploitation. The best long-term strategy may involve short-term sacrifices. For example, one exploration trial could lead to failure, but it educates the agent to not choose that action too often in the future. One of the ways to explore as well as exploit is using the ϵ-greedy exploration. ϵ-greedy is a policy that chooses a random action with probability ϵ. This means that the probability for exploration is ϵ. The agent chooses the optimal action with a probability of $1 - \epsilon$, which means nothing but the exploitation is done with $1 - \epsilon$ probability. For all the concepts that follow, the action a is selected using this ϵ-greedy exploration technique.

1.1.3 Q-Learning:

Q-Learning is based on the idea of a Q-function and a Q-table. The Q-function of a policy π, $Q^{\pi}(s, a)$, measures the expected return or discounted sum of rewards obtained from state s by taking the action a first and following policy π thereafter. An optimal Q-function $Q^*(s, a)$ is nothing but the maximum return that can be obtained starting from observation s, taking action a, $Q^*(s, a) = max_{\pi}\mathbb{E}[R_t|s, a_t = a, \pi]$, where π is a policy mapping sequences to actions: $Q : S \times A \mapsto \mathbb{R}$. The optimal Q function follows the Bellman Equation. This can be understood as follows: let us consider that the optimal value $Q^*(s', a')$ of the sequence s' at the next timestep is known for all possible actions a'. Under this assumption, the optimal strategy will obviously be to select the action a' maximising the expected value of $r + \gamma Q^*(s', a')$.

$$Q^*(s, a) = \mathbb{E}_{s' \sim P}[r + \gamma max_{a'} Q^*(s', a')|s, a] \tag{1.3}$$

This Q function is updated iteratively using the following equation: $Q_{i+1}(s, a) = \mathbb{E}[r + \gamma max_{a'} Q_i(s', a')|s, a]$. Such value functions are known to be converging to the optimal value function as $i \to \infty$. This is implemented using the following update equation:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma max_{a'} Q(s', a') - Q(s, a)) \tag{1.4}$$

Here, α is the learning rate. This equation makes sense, as we update the Q function according to the error $Q_{optimal} - Q(s, a)$. This is generally carried out for each step in an episode, until we reach the terminal state s.

Thus, Q-learning is a straightforward and robust method to form the Q-table for our agent. A significant drawback is that the table itself can be massive depending on the number of states and actions. Finding Q-values of all possible state-action pairs from a big Q-table is itself a computationally intensive task. The computation will be equally high for the save and update step of the Q-table. Hence, there is a need to compute Q values for a given state quickly. A natural solution is to use a function approximator for the Q-function, like a neural network.

1.1.4 Deep Q-Network (DQN):

We have established that we need to use a neural network for obtaining the various possible Q values for all possible actions from a given state (input to the network). The Q-function cannot be directly used for this purpose and is parameterised by the parameter θ. After obtaining the Q values, the best action can be selected with the maximum Q value. Since a neural network is used to approximate the Q value, this system is called a Q network. Generally, a deep neural network is used to approximate the Q value, and hence, the deep neural network is called a **deep Q network (DQN)**.
The new parameterised Q function is denoted as $Q_\theta(s, a)$. The parameter θ is initialised with random values, which give the sub-optimal Q function. After training the network for several iterations, optimal θ is found, leading to the optimal Q function. The first challenge here is training the network.

Replay Buffer:

RL does not have predefined datasets for training data. It learns from experience. Thus, the agent's experience is collected in a buffer D called *replay buffer*. A replay buffer is a *finite* database of transition information. This includes the present state s, the action a which leads the agent to the next state s', and the resulting reward r. Thus, the tuple (s, a, r, s') denotes the transition information stored in the buffer. This forms the training data for the DQN. This replay buffer has a limited size and can only store a fixed amount of the agent's experience. Once the buffer gets filled, the next entry overwrites the oldest entry of the buffer. This can be thought of as replacing the old experience with the new experience. While thinking in programming terms, the replay buffer can be considered a queue data structure following a first-in, first-out FIFO rule. We can see that the *transition data are stacked sequentially*. Thus, they are highly correlated and cannot be used directly as training data. So, random mini-batches are sampled from the buffer and are used for training the DQN.

Loss Function:

Q values are continuous, which essentially means that the DQN performs a regression task. Instead of a normal loss, mean squared error (MSE) is used to check how close predictions are to actual ground truth. Lower MSE indicates a better prediction. MSE is typically used for evaluating regression models, as lower MSE is directly related to a better fit. Hence, MSE is used as the loss function. MSE is basically taking the average squared difference between the target value and the predicted value. It is given by:

$$MSE = \frac{1}{K}\sum_{i=1}^{K}(y_i - \hat{y}_i)^2 \tag{1.5}$$

Here, the target value is given by y, and the predicted value is given by $\hat{y}$. K denotes the number of training samples in the dataset. In the case of DQN, the goal is to minimise the MSE between the target Q value and the predicted Q value. The target Q value is nothing but the optimal Q value given by the Bellman equation as follows:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P}[R(s, a, s') + \gamma max_{a'} Q^*(s', a')] \tag{1.6}$$

The loss (difference between target and predicted Q values) is given by:

$$L(\theta) = Q^*(s, a) - Q_\theta(s, a) \tag{1.7}$$

Here, expectation can be ignored and instead, it is approximated as the mean value got by sampling K number of transitions from the replay buffer. Thus, the loss becomes:

$$L(\theta) = r + \gamma max_{a'} Q(s', a') - Q_\theta(s, a) \tag{1.8}$$

We have seen above that the predicted Q value is found using the network parameterised by θ. Nevertheless, how to compute the Q value of the next state-action pair, i.e. $Q(s', a')$? The natural solution is to use the same parameterised DQN as used for computing the predicted values. The loss can now be written as:

$$L(\theta) = r + \gamma max_{a'} Q_\theta(s', a') - Q_\theta(s, a) \tag{1.9}$$

As said previously, a randomly sampled minibatch consisting of K transitions (s, a, r, s') from the replay buffer is used to train the DQN. The loss function used for this is the MSE, given as:

$$L(\theta) = \frac{1}{K} \sum_{i=1}^{K} (r_i + \gamma max_{a_i} Q_\theta(s'_i, a') - Q_\theta(s_i, a_i))^2 \tag{1.10}$$

This equation is commonly written with the target value represented by y where $(y_i = r_i + \gamma max_{a'} Q_\theta(s'_i, a_i)$. In the definition of y, we have not considered the case when s' is a terminal state. Hence, the definition of y_i is changed as follows:

$$y_i = \begin{cases} r_i & \text{if } s' \text{ is terminal} \\ r_i + \gamma max_{a'} Q_\theta(s'_i, a') & \text{if } s' \text{ is not a terminal} \end{cases}$$

Thus, the loss function is now written as:

$$L(\theta) = \frac{1}{K} \sum_{i=1}^{K} (y_i - Q_\theta(s_i, a_i))^2 \tag{1.11}$$

For training the DQN and finding the optimal parameter θ, gradient descent algorithm is used. The update step for the parameter is given as:

$$\theta = \theta - \alpha \nabla_\theta L(\theta) \tag{1.12}$$

Here, α is the learning rate, and $L(\theta)$ is the loss function.

Target Network:

There is one obvious flaw with the concept and the MSE formula discussed in the previous section. The optimal Q value (target) and the predicted Q value both depend on the same parameter θ. So, whenever one iteration (forward pass) is performed in the neural network, θ is updated to bring the $Q_\theta(s_i, a_i)$ closer to $r_i + \gamma max_{a'} Q_\theta(s'_i, a')$. During the next forward pass, the target Q value has changed due to the previously updated parameter θ. This makes the MSE unstable, and divergence is experienced during training. Clearly, we need to freeze the target Q value while computing the predicted Q value. Another neural network, called **the target network**, is used to compute the target Q value for the next state and action pair. This target network can be parameterised by θ' Thus, we now have two different networks, the primary network (DQN) for predicting the Q values, and the target network. The primary network uses gradient descent to find the optimal parameter θ, until then the target network is frozen. Once θ stabilizes during the training, the target network parameter θ' is updated by copying the primary DQN parameter θ. Thus, the loss function can now be written as follows:

$$L(\theta) = \frac{1}{K} \sum_{i=1}^{K} (r_i + \gamma max_{a'} Q_{\theta'}(s'_i, a') - Q_\theta(s_i, a_i))^2 \tag{1.13}$$

DQN- The Algorithm:

The DQN algorithm is given as follows[2]:

1. Initialize Replay Buffer D with finite capacity.
2. Initialize primary network parameter θ with random values.
3. Initialize target network parameter θ' by copying θ.
4. **for** $episode = 1, N$ **do**

[2]Playing Atari with Deep Reinforcement Learning, "Deep Reinforcement Learning with Python" by Sudarshan Ravichandran

(a) **for** $t = 0, T$ (for each step in episode) **do**

i. observe state s. Use ϵ-greedy exploration strategy: with probability ϵ select random action a_t, with probability $1 - \epsilon$ select action $a = max_a Q_\theta(s, a)$.

ii. Execute the selected action, observe reward r, and move to the next state s'.

iii. Store the transition information tuple (s, a, r, s') in the replay buffer D.

iv. Sample random minibatch of K transitions from the replay buffer D.

v. Compute target value

$$y_i = \begin{cases} r_i & \text{if } s' \text{ is terminal} \\ r_i + \gamma max_{a'} Q_\theta(s'_i, a') & \text{if } s' \text{ is not a terminal} \end{cases}$$

vi. Compute the loss, $L(\theta) = \frac{1}{K} \sum_{i=1}^{K} (y_i - Q_\theta(s_i, a_i))^2$

vii. Use gradient descent step on primary network parameter θ: $\theta = \theta - \alpha \nabla_\theta L(\theta)$

viii. Freeze target network parameter θ', after θ stabilizes, update θ' by copying θ.

end for

end for

1.1.5 DeepMind Atari games using DQN:

In 2013, DeepMind Technologies developed the DQN algorithm. They used it to train an agent to play several Atari 2600 games. Since images of the game were taken as observations, a Convolutional neural network (CNN) was used as the primary neural network. Since the input was 210 × 160-pixel images with a 128 colour palette, a basic preprocessing was applied to reduce the dimensionality. The CNN extracted features from the input images to create a feature map. After flattening the feature map, a feedforward network calculates the Q values of all the actions in the state. Pooling operation was not performed as it is only relevant for object detection/classification type of tasks.

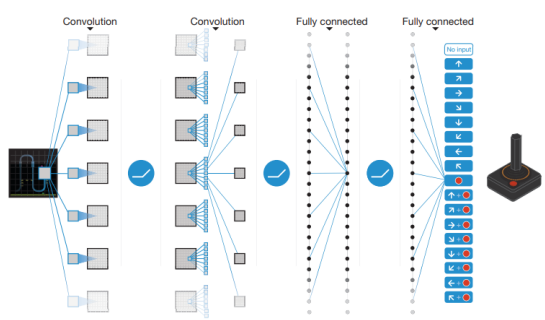

Figure 1.2: DQN with CNN used by DeepMind for training the agent for playing Atari games[3]

An ϵ-greedy policy with ϵ= 0.05 was used. The results obtained were deemed state-of-the-art for three of the seven games the agent was tested on, without changing any architecture or hyperparameters in between. It achieved closed to human performance in one and was far from it in the rest three (they required the network to find a strategy that lasted for many time steps).

1.1.6 Improvements to DQN

The DQN algorithm has some drawbacks. It overestimates the target Q value by selecting the maximum action value a'. This selected a' might be sub-optimal due to the presence of noise. Double DQN Algorithm was introduced to overcome this. Another con of DQN is that the random sampling while selecting the minibatches (training data) might not be the most optimal solution. DQN with prioritized delay is used to remove the randomness. Some other modifications of DQN include the duelling DQN, and the deep recurrent Q Network. Overall, DQN is a fundamental algorithm and has applications in navigation for autonomous robots, healthcare, algorithmic trading in finance.

[3]Human-level control through deep reinforcement learning